

Procrastination Prevention System

Introduction

Procrastination is a common challenge in productivity and focus management. This project addresses the issue by creating a Procrastination Prevention System that integrates user intention logging, screen activity monitoring, and behaviour analysis using machine learning. The system combines components for summarization, classification, and visualization to ensure users stay on track with their intended tasks.

Approach

The Procrastination Prevention System follows a modular design with the following core functionalities:

1. User Intention Logging:

- **Objective:** To collect and understand the user's intended tasks.
- **Implementation:**
 - Collected user input through the command line using argparse.
 - Parsed user intentions using a summarization model to simplify verbose entries.

2. Screen Monitoring:

- **Objective:** To periodically capture and analyze user screen activity.
- **Implementation:**
 - Utilized pyautogui to take screenshots at specified intervals.
 - Stored screenshots in a designated directory for further analysis.

3. Activity Classification:

- **Objective:** To classify screenshots into work-related or entertainment categories.
- **Implementation:**
 - Leveraged the facebook/bart-large-mnli model for zero-shot classification.
 - Labels used: “work-related” and “entertainment”.

4. Notifications:

- **Objective:** To alert users when their activities deviate from their logged intentions.
- **Implementation:**
 - Used the plyer library to send desktop notifications.

5. Analytics Dashboard:

- **Objective:** To visualize and analyze user behavior trends.
- **Implementation:**
 - Generated visualizations using pandas and matplotlib for:
 - Frequency of events (e.g., summarizations, notifications, classifications).
 - Timing of screenshots and activity classification results.
 - Data was read from a CSV file (analytics.csv) updated during runtime.

Models and Libraries Used

Models

1. **Summarization:**
 - **Model:** t5-small (from the Hugging Face Transformers library).
 - **Use Case:** To simplify user intentions into concise summaries.
2. **Zero-Shot Classification:**
 - **Model:** facebook/bart-large-mnli (from the Hugging Face Transformers library).
 - **Use Case:** To classify screenshots without requiring labeled training data.

Libraries

1. **General Libraries:**
 - argparse: Command-line argument parsing.
 - os: File and directory management.
 - time: Timestamp generation and delays.
 - logging: Capturing detailed logs for debugging and analytics.
2. **Machine Learning and NLP:**

- transformers: Loading pre-trained models for summarization and classification.
- 3. **Screen Monitoring:**
 - pyautogui: Capturing screenshots programmatically.
- 4. **Notifications:**
 - plyer: Sending desktop notifications.
- 5. **Data Analysis and Visualization:**
 - pandas: Data manipulation and analysis.
 - matplotlib: Plotting visualizations for analytics.

System Workflow

1. **Initialization:**
 - Collect user intentions through command-line input.
 - Parse the intention into a concise summary.
2. **Monitoring:**
 - Capture periodic screenshots of the user's activity.
 - Classify each screenshot into predefined categories.
3. **Behavior Analysis:**
 - Log activities and classifications in analytics.csv.
 - Provide real-time alerts via notifications when necessary.
4. **Visualization:**
 - Generate an analytics dashboard to display:
 - Event counts by type.
 - Screenshot capture trends.
 - Insights into user focus patterns.

Results and Observations

1. The summarization model effectively reduced verbose user inputs into actionable summaries.
2. Zero-shot classification provided adequate performance for basic classification tasks without requiring labeled datasets.
3. Real-time monitoring and notifications worked seamlessly to alert users about deviations.
4. The analytics dashboard enabled a high-level overview of user behavior and system performance.

Challenges Faced

1. **Screenshot Classification:**
 - Limited to textual labels rather than visual content analysis.
 - Could be improved with computer vision models for more accurate classification.
2. **System Performance:**
 - Screenshot capturing and classification can become resource-intensive.
3. **Data Integrity:**
 - Ensuring accurate logging and synchronization between processes (e.g., logging and analytics visualization).

Future Improvements

1. **Enhanced Computer Vision:**
 - Incorporate OCR (Optical Character Recognition) to analyze text within screenshots.
 - Use pre-trained vision models (e.g., YOLOv8) for more robust content classification.
2. **Real-Time Analysis:**
 - Implement real-time analysis of screen activity using streaming data pipelines.
3. **User Customization:**

- Allow users to define custom activity categories and labels.
- 4. **Improved Notification System:**
 - Introduce adaptive notifications based on user behavior patterns.
- 5. **Cloud Integration:**
 - Store analytics data in the cloud for remote access and extended storage.
 - Create a web-based dashboard for easier visualization.

Conclusion

The Procrastination Prevention System demonstrates the integration of natural language processing, computer vision, and analytics to tackle the challenge of staying focused. While the system is functional and provides valuable insights, future improvements in classification accuracy, system efficiency, and user experience can significantly enhance its effectiveness. This project serves as a strong foundation for building intelligent productivity tools.